arguments that really need to be references in `std::ref`. In this case, if you change the thread invocation to

```
std::thread t(update_data_for_widget,w,std::ref(data));
```

and then `update_data_for_widget` will be correctly passed a reference to `data` rather than a reference to a *copy* of `data`.

If you're familiar with `std::bind`, the parameter-passing semantics will be unsurprising, because both the operation of the `std::thread` constructor and the operation of `std::bind` are defined in terms of the same mechanism. This means that, for example, you can pass a member function pointer as the function, provided you supply a suitable object pointer as the first argument:

```
class X
{
public:
    void do_lengthy_work();
};

X my_x;
std::thread t(&X::do_lengthy_work,&my_x);
```


This code will invoke `my_x.do_lengthy_work()` on the new thread, because the address of `my_x` is supplied as the object pointer ❶. You can also supply arguments to such a member function call: the third argument to the `std::thread` constructor will be the first argument to the member function and so forth.

Another interesting scenario for supplying arguments is where the arguments can't be copied but can only be *moved*: the data held within one object is transferred over to another, leaving the original object "empty." An example of such a type is `std::unique_ptr`, which provides automatic memory management for dynamically allocated objects. Only one `std::unique_ptr` instance can point to a given object at a time, and when that instance is destroyed, the pointed-to object is deleted. The *move constructor* and *move assignment operator* allow the ownership of an object to be transferred around between `std::unique_ptr` instances (see